



KALIMIND

Autonomous Vulnerability Assessment Agent

LLM-Assisted Penetration Testing via Model Context Protocol (MCP)

Team KaliMind | BTech CSE (Cyber Security) | Dept. of Networking & Communications

SRM Institute of Science and Technology, Kattankulathur

R Aswin (RA2311030010258) • Deepthi Chand Mungara (RA2311030010260) • Srikar Bharath (RA2311030010227)

Faculty Guide & Panel Head

Dr. C.N.S. Vinoth Kumar

Panel Members:

Dr. C.N.S. Vinoth Kumar | Dr. P. Balaji
Srikanth | Dr. R. Nithya | Ms. Jasmine S

ABSTRACT

KaliMind is an LLM-Assisted Vulnerability Assessment Tool that bridges the reasoning power of Large Language Models (LLMs like Claude) with the execution capabilities of industry-standard penetration testing tools on Kali Linux. By leveraging the Model Context Protocol (MCP), KaliMind transforms traditional static scanning into an autonomous, context-aware penetration testing pipeline.

Unlike conventional scanners that rely on rigid signatures and produce false positives requiring manual validation, KaliMind enables an LLM to function as an active, reasoning penetration tester — autonomously chaining tools, interpreting results, adapting payloads, and reporting actionable findings.



LLM-Driven
Execution



Autonomous
Pipelining



Context-Aware
Exploitation



Authorized
Security Testing

PROBLEM STATEMENT



Manual & Time-Consuming

Traditional penetration testing requires skilled testers to manually chain tools, validate findings, and filter false positives — an exhaustive and slow process.



Rigid Signature-Based Scanning

Existing scanners run fixed scripts and flag issues based on pre-defined patterns, lacking intelligence to adapt to context or chain attack vectors autonomously.



Shortage of Skilled Experts

There is a severe global shortage of cybersecurity professionals capable of conducting full penetration testing cycles at speed and scale needed by modern organizations.



Siloed Tool Ecosystem

Penetration testing tools like nmap, Metasploit, Hydra, SQLMap etc. operate in isolation, requiring manual data transfer and interpretation between each stage.

VISION STATEMENT



To democratize cybersecurity by empowering organizations of all sizes with an intelligent, autonomous vulnerability assessment agent that thinks like an expert penetration tester — making secure infrastructure accessible, scalable, and actionable for a safer digital world.

Purpose

Enable autonomous, AI-driven vulnerability assessment to replace manual, error-prone penetration testing workflows.

Target Audience

Security teams, ethical hackers, academic researchers, and enterprises requiring scalable, repeatable security audits.

Long-Term Impact

Reduce time-to-detect critical vulnerabilities, lower cost of security assessments, and upskill future security professionals.

SDG Alignment

SDG 9 (Industry, Innovation & Infrastructure) and SDG 16 (Peace, Justice & Strong Institutions) via safer digital ecosystems.

SYSTEM ARCHITECTURE



Human Operator

Issues high-level task instructions



AI Agent (Claude LLM)

Formulates plan, calls tools, reads results



KaliMind MCP Client

kalimind_mcp.py — LLM to HTTP translation



Kali Backend Engine

kalimind_backend.py — Flask API + execution

kalimind_mcp.py — Intelligent Bridge

- Runs on the operator's local machine alongside Claude Desktop
- Registers all pentesting tools with MCP (nmap, hydra, metasploit...)
- Translates LLM tool invocations into HTTP REST requests
- Teaches the LLM what tools exist, parameters, and when to use them

kalimind_backend.py — Execution Engine

- Runs directly on the Kali Linux VM or Docker container
- Flask REST API spawning isolated subprocess shells per tool
- Async process handling with configurable timeout management
- Executes: nmap, gobuster, hydra, metasploit, sqlmap, john, enum4linux

CORE CAPABILITIES



Reconnaissance & Web Scanning

- `nmap_scan` — Dynamic port & version enumeration
- `gobuster_scan` & `dirb_scan` — Hidden endpoint discovery
- `nikto_scan` — Web server vulnerability audit
- `wpscan_analyze` — WordPress CVE & plugin analysis



Exploitation & Vulnerability Assessment

- `sqlmap_scan` — Automated SQL injection & DB takeover
- `metasploit_run` — Autonomous exploit module execution
- Payload generation & RHOSTS/LHOST configuration
- Full exploitation feedback loop into LLM context



Enumeration & Password Cracking

- `enum4linux_scan` — SMB/Windows share enumeration
- `hydra_attack` — Dictionary & brute force on login portals
- `john_crack` — Hash cracking for discovered credentials
- Context-aware wordlist generation by the LLM



Absolute Control

- `execute_command` — Raw shell access on Kali machine
- Supports `curl`, `netcat`, `jq`, and custom Python scripts
- LLM can chain any tool not explicitly wrapped
- Real-time stdout/stderr piped back to LLM context

PRODUCT BACKLOG & EPICS

Epic ID	Epic Name	Priority	Sprint	User Stories
EP-01	MCP Server & Backend Setup	Critical	1	US-01: Configure Flask API US-02: Tool endpoint registration US-03: Subprocess management
EP-02	Reconnaissance Module	High	1-2	US-04: nmap integration US-05: gobuster & dirb US-06: Result parsing & LLM feed US-10: WPScan WordPress scanner US-11: Nikto web vulnerability scan
EP-03	Exploitation Engine	High	2-3	US-07: Metasploit automation US-08: SQLMap integration US-09: Payload customization
EP-04	Cracking & Enumeration	Medium	3	US-12: Hydra attack module US-13: John the Ripper crack US-14: enum4linux SMB scan
EP-05	LLM Agent Intelligence	Critical	1-4	US-15: Tool chaining logic US-16: Context-aware decisions US-17: Report generation
EP-06	Security & Compliance	High	4	US-18: Authentication layer US-19: Audit logging US-20: Safe architecture guide

SPRINT PLAN

Sprint 1

Weeks 1-2

Goal:

Foundation & Recon Setup

Key Tasks:

- Set up Flask backend on Kali VM
- Build MCP client (kalimind_mcp.py)
- Integrate nmap, gobuster, dirb, nikto
- Test LLM ↔ MCP ↔ Backend communication

Sprint 2

Weeks 3-4

Goal:

Exploitation & SQLMap

Key Tasks:

- Integrate Metasploit headless execution
- Add SQLMap automated scan module
- Implement timeout & async process mgmt
- LLM context-aware tool chaining

Sprint 3

Weeks 5-6

Goal:

Cracking & Enumeration

Key Tasks:

- Add Hydra brute force & John cracker
- Integrate enum4linux for SMB recon
- Build wordlist generation from LLM
- End-to-end attack chain testing

Sprint 4

Weeks 7-8

Goal:

Security, Reporting & Polish

Key Tasks:

- Implement API authentication layer
- Build automated report generator
- Security hardening & audit logging
- Final demo, documentation & submission

REQUIREMENTS

Functional Requirements

- FR-01: System shall allow LLM to invoke penetration testing tools via MCP protocol
- FR-02: System shall execute nmap, gobuster, nikto, dirb scans autonomously
- FR-03: System shall run Metasploit exploit modules with configured payloads
- FR-04: System shall perform SQL injection testing via SQLMap on discovered URLs
- FR-05: System shall execute brute-force attacks via Hydra on login portals
- FR-06: System shall crack password hashes using John the Ripper
- FR-07: System shall enumerate Windows/SMB environments via enum4linux
- FR-08: System shall allow raw shell command execution on the Kali machine
- FR-09: System shall feed all tool output back into the LLM's context window
- FR-10: System shall generate a structured vulnerability assessment report

Non-Functional Requirements

Performance: Tool executions must return results within configured timeout (default: 5 min)

Reliability: Async process handling must not deadlock or crash backend on long-running attacks

Security: API must be restricted to local/VPN networks; no internet exposure

Scalability: Architecture must support multiple concurrent scanning sessions

Usability: LLM must receive clear, parseable tool output for downstream reasoning

Maintainability: Tools are modular; new pentesting tools can be added without refactoring

Portability: Backend deployable on Kali VM, Docker container, or bare metal

PRODUCT ROADMAP

Phase 1 — Foundation

Release: v0.1

- MCP setup
- nmap + gobuster
- Backend API

Phase 2 — Core Tools

Release: v0.5

- Metasploit
- SQLMap
- Nikto/WPScan

Phase 3 — Attack Chain

Release: v0.8

- Hydra & John
- enum4linux
- Chaining logic

Phase 4 — Production

Release: v1.0

- Auth layer
- Reporting
- Hardening

Future Vision



v2.0: Cloud-native multi-target scanning with dashboard UI



v2.5: ML-based false positive filtering & risk scoring



v3.0: Multi-LLM orchestration for parallel attack threads



v3.5: Compliance mapping (OWASP, NIST, ISO 27001)

TEAM & AGILE ROLES



Product Owner

R Aswin

RA2311030010258

- Defines product vision & roadmap
- Maintains and prioritizes product backlog
- Accepts sprint deliverables
- Liaison with panel & stakeholders



Scrum Master

Deepthi Chand Mungara

RA2311030010260

- Facilitates sprint planning & daily standups
- Removes blockers & impediments
- Maintains Agile board (Jira/Trello)
- Ensures team follows Scrum principles



Developer

Srikar Bharath

RA2311030010227

- Develops kalimind_mcp.py & backend
- Integrates pentesting tools via Flask API
- Writes unit tests & integration tests
- Documents API endpoints & usage

Agile Framework: 2-week Sprints | Daily Standups | Sprint Reviews | Retrospectives | Agile Board: Jira / GitHub Projects

SDG ALIGNMENT & IMPACT



SDG 9 — Industry, Innovation & Infrastructure

KaliMind introduces a disruptive AI-powered approach to cybersecurity — enabling innovation in automated infrastructure auditing and building resilient digital ecosystems accessible to all organizations.



SDG 16 — Peace, Justice & Strong Institutions

By enabling proactive vulnerability detection, KaliMind empowers institutions to protect sensitive data, prevent cyber attacks on public systems, and maintain secure, trustworthy digital governance.



SDG 17 — Partnerships for the Goals

Open-source architecture encourages collaboration between academia, industry, and security researchers. Designed for use in educational institutions to train next-gen cybersecurity professionals.

SRMIST Theme Alignment: Disruptive Technologies

70%

Reduction in
Manual Effort

3x

Faster Vulnerability
Detection

100%

Authorized
Use Cases Only

10+

Integrated
Pentesting Tools

SECURITY, ETHICS & DISCLAIMER



No Public Exposure

The Kali backend API must NEVER be exposed to the public internet. Bind only to local/VPN networks. Use SSH tunneling for remote access.



Authentication Required

Current MVP has no authentication layer. Production deployments must implement token-based auth before using in any real environment.



Authorized Use Only

KaliMind must only be used against systems you own or have explicit, written authorization to test. Unauthorized testing is illegal worldwide.



Isolated Environment

Always run the Kali backend in a Host-Only or NAT VM. Never on a production machine. Use Docker containers for additional isolation.



THANK YOU

KaliMind — Autonomous Vulnerability Assessment Agent

R Aswin • Deepthi Chand Mungara • Srikar Bharath

BTech CSE (Cyber Security) | Dept. of Networking & Communications
SRM Institute of Science and Technology, Kattankulathur

Faculty Guide & Panel Head: Dr. C.N.S. Vinoth Kumar | Panel Members: Dr. C.N.S. Vinoth Kumar • Dr. P. Balaji Srikanth • Dr. R. Nithya • Ms. Jasmine S